QSVD Optimization Report: Naive Autograd vs. Projection Hooks

1. The Core Mathematical Problem

The goal of Gradient-Based QSVD is to calculate the sensitivity (importance) of the singular vectors **U** and **V** when the layer's weights are compressed.

Mathematically, calculating the gradient of the loss with respect to the singular values requires computing the projection of the exact Weight Gradient Matrix (**G_w**) onto the singular vectors: **Importance = U^T * G_w * V**

2. The Original Paper Approach (Naive Autograd)

The original code in the QSVD paper relies entirely on PyTorch's default backpropagation engine to calculate **G_w**.

- **How it works:** It does a standard forward pass, calls `loss.backward()`, and waits for PyTorch to physically populate `q_proj.weight.grad` with the exact 4096x4096 gradient matrix.
- **The Fatal Flaw:** PyTorch calculates those weight gradients via the outer product of the layer's inputs (**X**) and output-gradients (**grad_Y**), i.e., **G_w = X^T * grad_Y**. Physically instantiating this massive 4096x4096 matrix for Q, K, and V matrices across all 32 layers demands enormous amounts of contiguous GPU memory.
- **The Symptoms:**

- **Memory:** Causes massive VRAM spikes and memory fragmentation, leading to unavoidable `CUDA Out of Memory` crashes on standard GPUs.
- **Speed:** Because the GPU allocator is struggling to find contiguous blocks for these massive matrices, calculation time crawls to a miserable **~19 seconds per batch**.

3. Our Solution: Hook-Based Activation Projections

We refactored the code to use a mathematically equivalent but computationally superior approach. By leaning into the associative property of matrix multiplication, we can calculate the final scalar importance scores *without ever physically instantiating the massive weight gradient matrix*.

The Math Trick: U^T * (X^T * grad_Y) * V === (X * V) * (grad_Y * U)

- **How it works:** We first set `requires_grad = False` for all weight matrices in the model. This explicitly stops PyTorch from trying to allocate the massive gradient tensors. Instead, we attach dynamic PyTorch **hooks** directly to the PyTorch computation graph.
- **On the fly execution:** During the backward pass, our hooks instantly intercept the raw Input Activations (**X**) and the raw Output Gradients (**grad_Y**). We multiply them directly by the compact singular vectors (**U** and **V**), add up the scores, and immediately delete the raw tensors.
- **The Results:**
- **Memory:** Memory usage stays completely flat. Because **G_w** is never physically built in VRAM, the out-of-memory crashes are completely cured.
- **Speed:** Removing the memory allocation bottleneck drops the compute time from **~19 seconds** to just **~1.6 seconds** per batch.

4. End-to-End Comparison Summary

Metric	Original Paper Implementation	Our Projection Implementation	Improvement
Weight Gradients	Fully Allocated (`requires_grad=True`)	Disabled (`requires_grad=False`)	Massive VRAM Savings
Compute Method	Post-processing 4096x4096 arrays	On-the-fly hook interception	Eliminated fragmentation
Time per Batch	~19.3 seconds	~1.6 seconds	**~12x Faster**
System Stability	Severe risk of OOM on 32GB GPUs	Flat & stable memory footprint	**100% OOM Free**